Chapter 5: Logical Operators

Booleans and Comparisons gave expressions truth values. Now we can combine those values: do both requirements hold, does either one hold, or is a condition false? Logical operations compose boolean results. (1)

This version adds `and`, `or`, and `not`, with symbolic aliases. The parser establishes their precedence. The evaluator decides when it needs to evaluate both operands and when the left operand alone determines the result.

Breaking Changes

The names `and`, `or`, and `not` are now reserved operator keywords. Rename variables that used those exact names. Longer identifiers such as `android`, `origin`, and `notable` still work because the patterns require word boundaries.

Booleans, comparisons, conversions, and `type()` carry forward unchanged. Print still requires parentheses; semicolons still separate statements. No statement form changes. Logical operators require boolean operands rather than converting numbers or strings implicitly. Use `boolean(value)` when a conversion is intended.

Combining Comparisons

```
__input = "21";
answer = number(input()) * 2;
in_range = answer >= 40 and answer <= 50;
print(in_range);
print("Captured: " + __output);
```

The output is `true` followed by `Captured: true`. Each comparison produces a boolean. The `and` expression combines them, and assignment stores the result under `in_range`. These are ordinary expressions, not a new statement form.

The code is in `chapter-05-logical-operators/code`. From that directory:

```
./vertex example.v < /dev/null
bash run-tests.sh
```

The example retains the earlier comparisons and conversions, then exercises both logical-operator spellings. Its skipped input call does not read from the keyboard; the test script checks the complete output.

Two Spellings, One Token

Logical operators have word and symbolic spellings:

Source spelling	Token tag	Meaning
`and`, `&&`	`and`	Both operands must be true
`or`, `||`	`or`	At least one operand must be true
`not`, `!`	`not`	Reverse the truth value

The tokenizer does the normalization. These are the actual patterns:

```
(r"!=", "!="),
(r"and\b|&&", "and"),
(r"or\b|\|\|", "or"),
(r"not\b|!", "not"),
```

The `!=` pattern comes first. Otherwise its initial `!` could become a `not` token, leaving an assignment token behind. The word boundaries keep `android`, `origin`, and `notable` from being split into operators and leftover text.

After tokenization, the parser has no reason to distinguish `&&` from `and`. They share one grammar rule and one abstract syntax tree (AST) tag. This is also why the precedence of `!` is the same as `not` here, even if another language gives `!` different rules.

Truth Tables and Operand Types

Left	Right	`and`	`or`
`false`	`false`	`false`	`false`
`false`	`true`	`false`	`true`
`true`	`false`	`false`	`true`
`true`	`true`	`true`	`true`

`not true` is false, and `not false` is true. These are boolean operations on truth values. (1) Vertex requires every operand it actually evaluates for a logical operation to be a boolean. It does not interpret zero as false or nonempty text as true. `not 0` and `true and "yes"` raise type errors.

Logical operations always return booleans, not one of several possible operand types. Comparisons are a useful way to produce their operands:

```
score = 42;
accepted = score >= 40 and not score > 50;
print(accepted);
```

This prints `true`. The next section explains why the `not` applies to the comparison rather than directly to the number stored in `score`.

Precedence Is a Parser Decision

Precedence specifies how an expression is grouped when parentheses do not make that grouping explicit. (2) Vertex uses this order, from weakest to strongest:

Level	Operators
1	`or`, `||`
2	`and`, `&&`
3	`not`, `!`
4	`==`, `!=`, `<`, `<=`, `>`, `>=`
5	`+`, `-` (binary)
6	`*`, `/`
7	Unary `-`

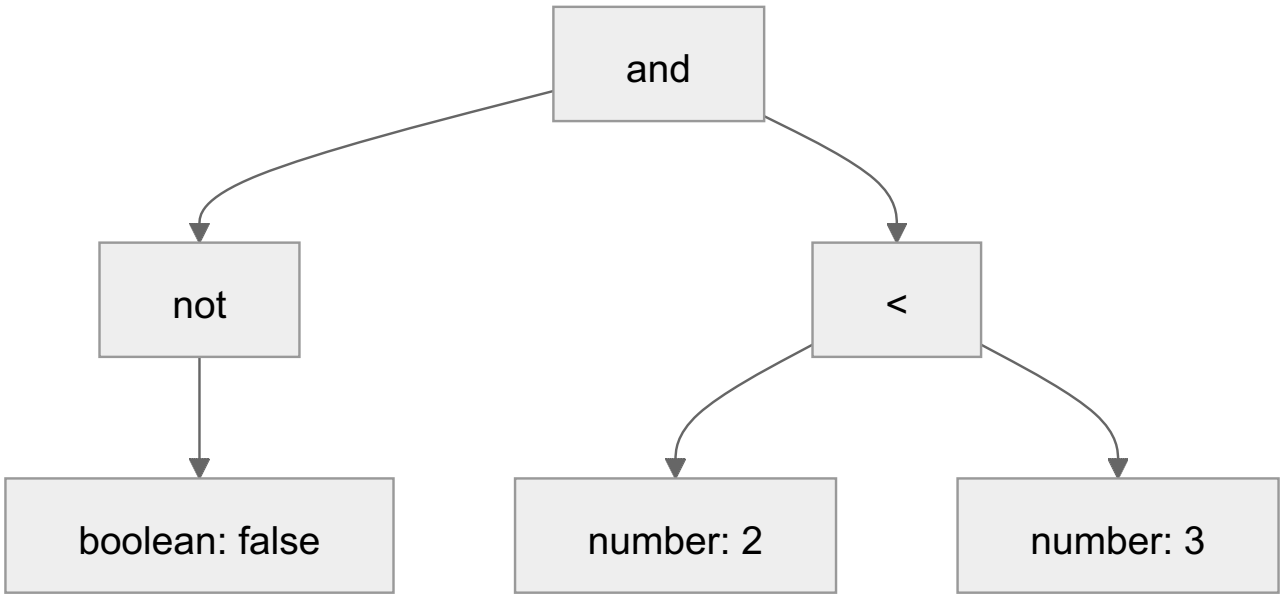
For example, `not x == y` means `not (x == y)`. Likewise, `a or b and c` means `a or (b and c)`. Parentheses can request another grouping. Repeated negation works: `!!true` and `not not true` both produce true.

The relevant Extended Backus-Naur Form (EBNF) rules are:

```
expression ::= logic_or
logic_or ::= logic_and { "or" logic_and }
logic_and ::= logic_not { "and" logic_not }
logic_not ::= "not" logic_not | comparison
comparison ::= arithmetic_expression [ compare_op arithmetic_expression ]
compare_op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
arithmetic_expression ::= term { ("+" | "-") term }
```

Only normalized word-form tags appear in these rules. Braces allow repetition; brackets make the comparison suffix optional. The existing `term`, `unary`, and `factor` rules continue below arithmetic. Boolean literals join the factor alternatives. A parenthesized factor calls `parse_expression` again, allowing the complete precedence hierarchy inside the parentheses.

The tree for `not false and 2 < 3` records the grouping:



Following the Parser

The existing `parse_arithmetic_expression` and `parse_comparison` stay in place. The public `parse_expression` now enters through `parse_logic_or`, the weakest level. Each level asks the next stronger level to recognize its operands.

For conjunction, the code is:

```
def parse_logic_and(tokens):
    # Each operand comes from the next tighter precedence level. The loop
    # constructs a left-associated tree without evaluating either operand.
    left, tokens = parse_logic_not(tokens)
    while tokens[0]["tag"] == "and":
        right, tokens = parse_logic_not(tokens[1:])
        left = {"tag": "and", "left": left, "right": right}
    return left, tokens
```

The loop makes `a and b and c` left-associated. Negation uses recursion rather than a binary loop because it has one operand and may repeat before that operand. None of these functions executes the expression. They only return nodes and unused tokens.

The comparison rule permits one comparison operator, not a chain. `1 < 2 < 3` is rejected. Write `1 < 2 and 2 < 3` instead. An explicitly nested comparison, such as `(1 < 2) == true`, is valid. Parentheses settle the grouping; they do not remove type requirements, so `(1 < 2) < 3` still fails during evaluation.

Short-Circuit Evaluation

For `and`, a false left operand already determines the answer. For `or`, a true left operand does the same. Short-circuit evaluation skips the right operand in those cases. (3) This changes which operations run, not just how quickly a result is produced.

```
divisor = 0;
safe = divisor != 0 and 10 / divisor > 2;
print(safe);
```

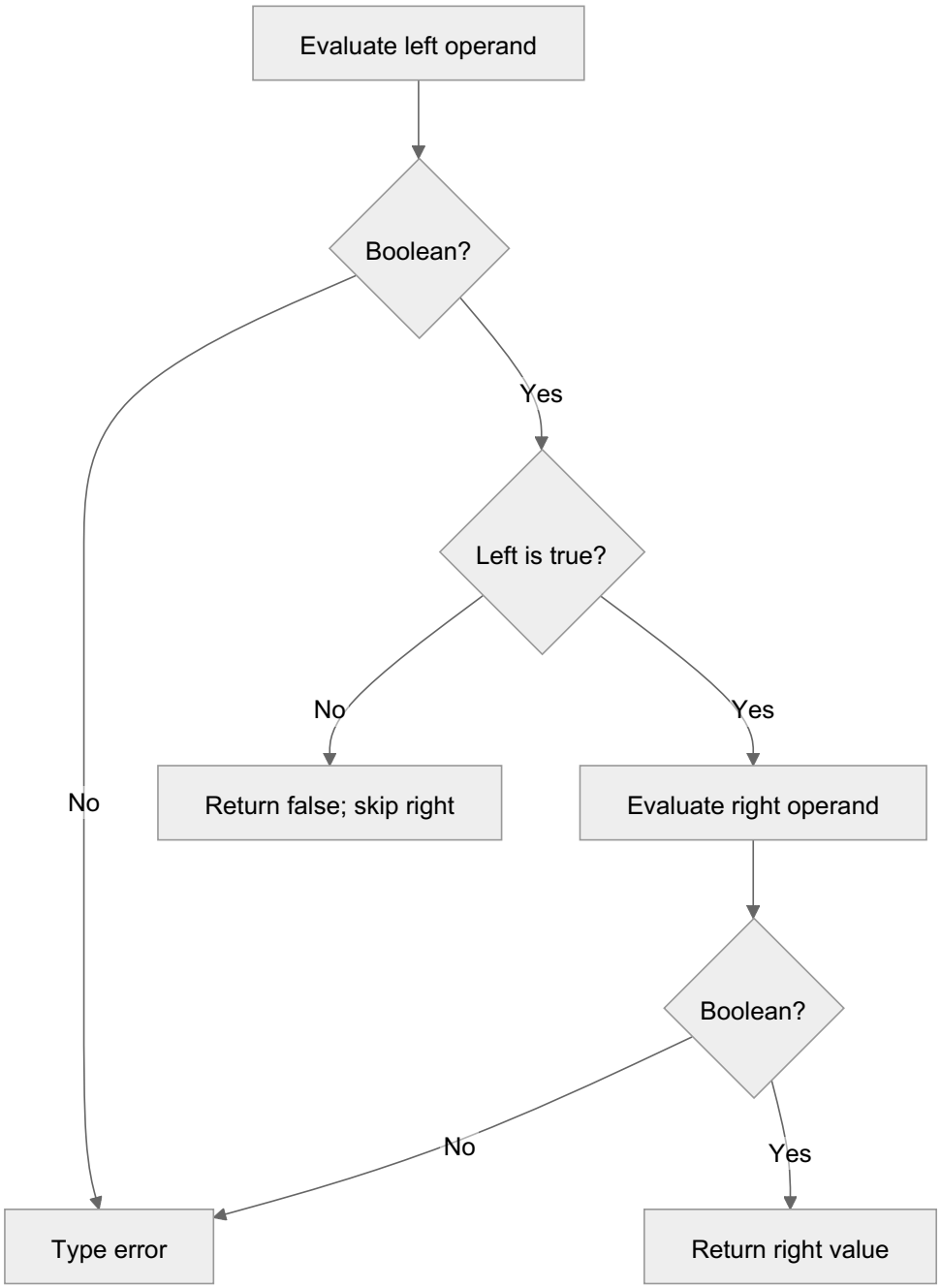
This prints `false`. The division never runs. Reversing the two operands would raise a division-by-zero error before the evaluator reached the guard.

The decision can be summarized without referring to source spelling:

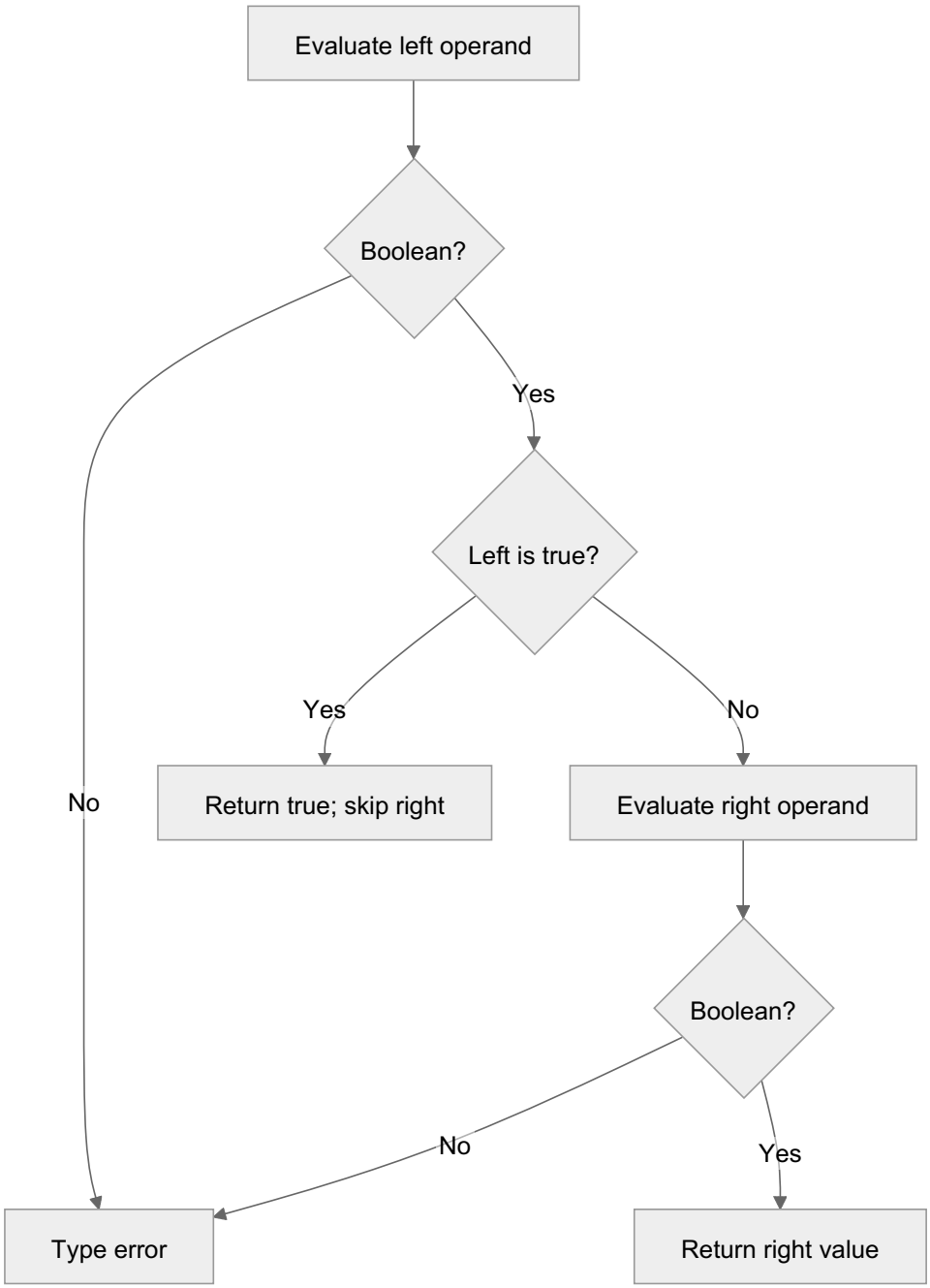
Operator	Left value	Next action
`and`	`false`	Return `false`; skip right
`and`	`true`	Evaluate right; require boolean
`or`	`true`	Return `true`; skip right
`or`	`false`	Evaluate right; require boolean

In either operator, an evaluated non-boolean left operand raises a type error. When the right side is needed, it must also produce a boolean. Its value is then the result of the whole logical expression.

For `and`, the false branch returns without visiting the right subtree:



For `or`, it is the true branch that can return immediately:



That is why the evaluator cannot handle `and` like an arithmetic operator by evaluating both children up front. The decision to evaluate the right child belongs inside the operator's evaluation rule.

Skipped Input and Skipped Errors

Input makes the effect visible:

```
__input = "ready";
result = false && input() == "ready";
print(result);
print(__input);
```

The output is `false` followed by `ready`. The input call did not run, so it did not consume or clear the supplied value. Replacing the left operand with true would run the input call and leave `__input` empty.

Short-circuiting also skips runtime errors. `true or missing` evaluates to true without looking up `missing`. `false and 42` evaluates to false without checking the skipped number's type. These are consequences of runtime checking, not a general permission to use numbers as logical operands.

Syntax must still be valid everywhere. The tokenizer and parser process the whole program before evaluation. `false and (1 +)` is rejected even though its right side would not be evaluated. An invalid escape in a skipped string is likewise still a tokenizer error.

Tests and Error Boundaries

The code retains the tests from Booleans and Comparisons, including conversions and `type()`. Logical-operator tests compare the ASTs produced by both spellings and exercise the complete truth tables. Mock input calls check that skipped operands do not read and needed operands read exactly once.

Program fragment	Result or error	Stage
`x = 1 + 2 * 3 == 7`	Stores `true`	Evaluation
`x = true & false`	Unsupported single &	Tokenization
`x = true and`	Missing operand	Parsing
`x = 1 < 2 < 3`	Extra comparison operator	Parsing
`x = not 1`	Non-boolean logical operand	Evaluation
`x = "1" < 2`	Mixed string and number operands	Evaluation
`x = false and missing`	Stores `false`	Evaluation

The runner still reports failures as Python tracebacks. The language does not yet have an assertion statement or conditionals. Tests use Python assertions and the existing `__input`/`__output` behavior; they do not require new Vertex syntax.

Considerations

1. Under what conditions can you swap the operands of `and` without changing observable behavior? What changes when either operand reads input?
2. Would rejecting non-boolean operands before execution improve this language? What information would a checker need about identifiers?
3. Why might a language give `!` higher precedence than comparisons? What does Vertex gain or lose by treating it exactly like `not`?
4. What extra semantics would be needed to support `low < input_value < high` as a comparison chain rather than rejecting it?

Practice

Answers to the practice questions follow this section.

1. Run `example.v`, then replace each word-form logical operator with its symbolic spelling. Confirm that the results are unchanged.
2. Draw the tree for `not false or true and false`. Add parentheses to change its result and draw the new tree.
3. Write a program that accepts supplied numeric input and checks whether it is between 10 and 20, inclusive. Check `__output` from Python.
4. Add a test showing that `true or input() == "yes"` leaves `__input` unchanged. Add another showing when the input must be consumed.
5. Add tests distinguishing the syntax error in `false and (1 +)` from the skipped runtime error in `false and 1 / 0 == 0`.
6. Explain why `true == 1` is false even though the host language uses Python booleans. Locate the evaluator check that makes this a Vertex decision.

Answers to the Practice Questions

1. Replace `and` with &&, `or` with `||`, and `not` with `!`. The tokenizer normalizes each pair to the same tag, so the parsed operations and printed results stay the same. Do not replace text inside string literals or comments as though it were an operator.

2. The root is `or`. Its left child is `not false`; its right child is `true and false`. The result is `true`. `(not false or true) and false` moves `and` to the root and produces `false`.

3. One inclusive-range check is:

```
from evaluator import evaluate_source

_, environment = evaluate_source(
    'n=number(input());print(n>=10 and n<=20)', {"__input": "15"})
assert environment["__output"] == "true"
```

Inputs `10` and `20` pass; `9` and `21` do not.

4. The right operand of `or` is needed only when the left operand is false:

```
from evaluator import evaluate_source

_, skipped = evaluate_source(
    'answer=true or input()=="yes"', {"__input": "yes"})
_, consumed = evaluate_source(
    'answer=false or input()=="yes"', {"__input": "yes"})
assert skipped["__input"] == "yes"
assert consumed["__input"] == ""
assert skipped["answer"] is True and consumed["answer"] is True
```

5. `false and (1 +)` raises `SyntaxError` during parsing, before there is a chance to short-circuit. `false and 1 / 0 == 0` parses successfully and evaluates to `false` without division. Separate tests should require the first error and check the second result without expecting an error.

6. The evaluator explicitly keeps booleans out of numeric equality. Python's own `True == 1` would be true, but Vertex does not delegate this decision unchanged to that host-language rule.

References

1. Wikipedia contributors. "Boolean data type." Wikipedia. https://en.wikipedia.org/wiki/Boolean_data_type
2. Wikipedia contributors. "Order of operations." Wikipedia. https://en.wikipedia.org/wiki/Order_of_operations
3. Wikipedia contributors. "Short-circuit evaluation." Wikipedia. https://en.wikipedia.org/wiki/Short-circuit_evaluation